

Documentación modelos YOLOv8

26 de febrero del 2024.

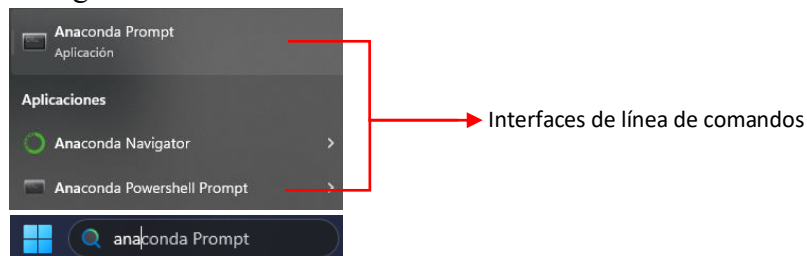
1. Pasos iniciales:

1.1 Verificación de Anaconda:

- En el sistema operativo debe estar instalado el software para la gestión de paquetes **Anaconda**. De lo contrario, en el sitio web oficial se proporcionan distintas versiones para su instalación:



- Verificación de Anaconda: Para verificar si Anaconda se encuentra en el sistema operativo, en la barra de tareas se debe ingresar el nombre. Si está instalado, aparecerán las aplicaciones ejecutables disponibles, incluyendo las herramientas de línea de comandos y la aplicación grafica “Anaconda Navigator”.



- ### 1.2 Modificación del archivo “environment.yml”:
- Este archivo contiene la lista de todas las dependencias que el proyecto necesita para funcionar

correctamente. Para personalizar el entorno al sistema operativo Windows, se debe seguir los siguientes pasos en la carpeta principal del proyecto:

- Cambio de la ruta prefix: En el archivo environment.yml, en el campo “prefix”, normalmente ubicado en la última línea de configuración, se especifica la ruta en el software donde se instalará el entorno virtual. Es importante el cambio de la ruta a un espacio de trabajo dedicado para los proyectos, se debe evitar proporcionar rutas con espacios en blanco o donde no exista permisos de acceso:

Advertencia: La ruta prefix debe ser absoluta, es decir, debe comenzar desde la raíz del sistema de archivos.

```
ultralitics==8.0.220
youtube-dl==2020.12.2
prefix: C:\Users\jemss\anaconda3\envs\pruebayolo
```

- Modificación opcional | Cambio del nombre del entorno virtual: En el campo “name”, normalmente ubicado en la primera línea de configuración, se especifica el nombre que se le da al entorno virtual para su procesamiento. Es importante que el nombre sea único entre los entornos virtuales existentes para evitar confusiones en la activación.

```
C:\Users\matrix> pruebayolo > proyecto_yolo > environment.yml
Click here to ask Blackbox to help you code faster
1 name: pruebayolo
2 channels:
3   - pytorch
4   - defaults
5 dependencies:
6   - blas=1.0
7   - brotli-python=1.0.9
8   - bzip2=1.0.8
9   - ca-certificates=2023.08.22
10  - certifi=2023.11.17
```

2. Iniciando el proceso con una línea de comandos: Para interactuar directamente con el entorno virtual, es necesario acceder a Anaconda Prompt.

Con el fin de recrearlo en el sistema, se debe ejecutar el comando **conda create -f ruta/del/directorio**, especificando la ruta donde se encuentra el archivo environment.yml con las configuraciones previas realizadas anteriormente:

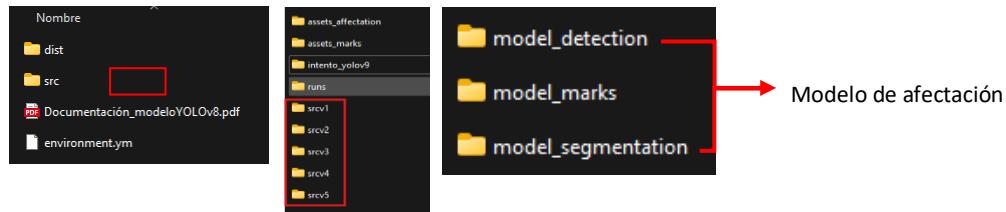
```
(base) C:\Users\jemss>conda create -f ruta/del/directorio/environment.yml
```

- Una vez recreado, es necesario activarlo: Proceso que configura la terminal para que cualquier paquete que se instale o cualquier script que se ejecute se haga dentro del entorno virtual. Es necesario escribir el comando **conda activate <nombre_del_entorno>** con el nombre que se encuentra en el campo “name” del archivo environment.yml:

```
Anaconda Prompt
(base) C:\Users\jemss>conda activate nombre_del_entorno
```

3. Preparación del entorno para la ejecución de scripts de producción:

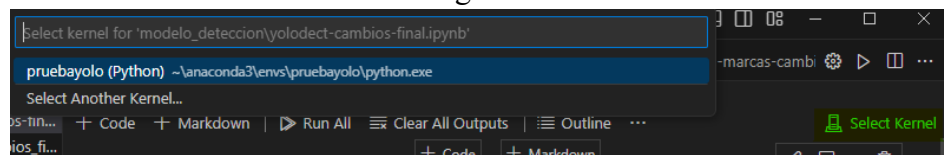
3.1 Ubicación de los scripts: En la carpeta raíz del proyecto, se encuentra una subcarpeta llamada 'src', que alberga las distintas versiones de los scripts. Se recomienda seleccionar la versión más reciente. Dentro de esta, se incluye el modelo de afectación para el modo de detección y segmentación, así como el modelo de marcas, en el interior de estas, se proporciona los scripts de ejecución y los entrenamientos correspondientes.



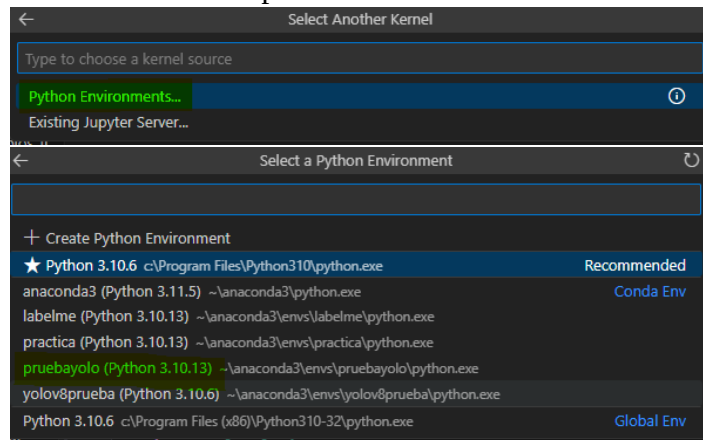
3.2 Selección de un IDE: Para poder ejecutar los scripts, se debe utilizar un entorno de desarrollo integrado (IDE), en este caso se utiliza VsCode con Jupyter:

[Visual Studio Code - Code Editing. Redefined](#) | [Project Jupyter](#) | [Home](#) | [colab.google](#)

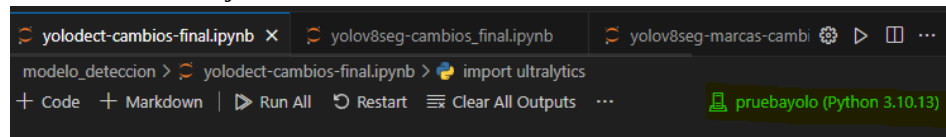
- Una vez abierto el script en el IDE. Al seleccionar el apartado “select kernel” se abrirá una ventana emergente:



- En la ventana emergente, se muestran los nombres de los entornos virtuales utilizados con frecuencia. En caso de que no aparezca el que se requiere, se debe seleccionar la opción “select another kernel” y luego en la alternativa “python enviroments”, donde se desplegará una lista; navega hasta encontrar el nombre del entorno virtual configurado previamente, selecciónalo con el puntero del ratón:

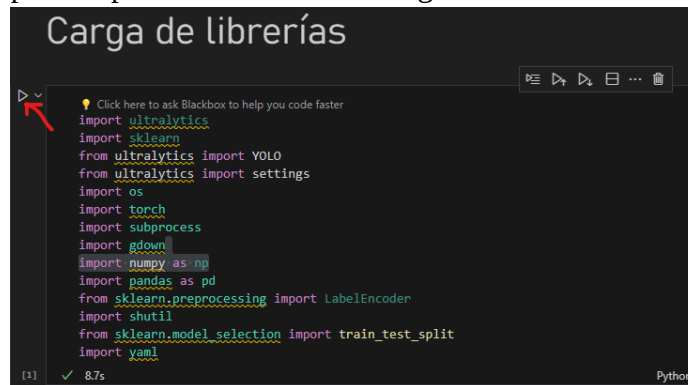


- Se debe ver reflejado de esta manera:



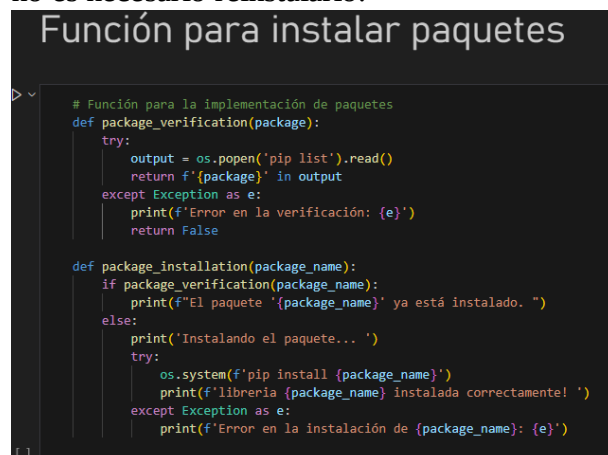
4. Inicialización del proyecto: Es fundamental que todas las bibliotecas utilizadas en el proyecto estén operativas, ya que ofrecen varias funcionalidades para la ejecución correcta del script:

- Debe verificarse si el entorno virtual está activo. Si no es así, en el ítem 3.2 se encuentran las instrucciones.
- En la sección “Carga de librerías”, debe seleccionar el botón de ejecución. Este botón generalmente se representa con una flecha y se encuentra en la parte superior del editor de código.



5. Función para instalar las dependencias: Ejecute la celda correspondiente a las funciones que revisarán el sistema y determinarán qué paquetes se necesita instalar.

Advertencia: Es importante tener en cuenta que una vez que un paquete está instalado, no es necesario reinstalarlo.



5.1 Instalación de recursos: Para poder utilizar los modos de YOLOv8, se necesita instalar la biblioteca Ultralytics y PyTorch:

Instalación de recursos para YOLO

Librería Ultralytics

```
> ~  
{ }  
package_installation('ultralytics') # Instalación de la biblioteca Ultralytics  
#ultralytics.checks()
```

- Para la instalación de PyTorch, es necesario verificar la compatibilidad del sistema operativo con el marco de trabajo de aprendizaje automático. La celda proporcionará información sobre la tarjeta gráfica del sistema e indicará qué opción se debe utilizar durante la instalación.

PyTorch

```
> ~  
{ }  
#Función para determinar si tiene GPU Cuda para instalación de PyTorch  
if torch.cuda.is_available():  
    device = torch.device("cuda")  
    print(f'CUDA: {torch.cuda.get_device_name(0)}')  
else:  
    try:  
        command = 'wmic path win32_videocontroller get caption'  
        device = subprocess.check_output(command, shell=True, universal_newlines=True)  
        print(f'Tarjeta Gráfica: {device.strip()}. \n Advertencia: Debe utilizar la CPU para instalación de PyTorch! ')  
    except subprocess.CalledProcessError as e:  
        print(f'Error al ejecutar el comando: {e}')
```

- Para instalar la versión de PyTorch que corresponde a los requisitos computacionales, es necesario ingresar a la página oficial, y ubicarse en la sección “get Started” o acceder al enlace directo con el puntero del ratón en “Versión PyTorch” en la sección que proporciona la información:

Dirección para instalar mediante el comando la versión de PyTorch en función de los requerimientos computacionales [Versión PyTorch](#)

- Es fundamental seleccionar las características que se ajusten al sistema operativo; estas se mostrarán resaltadas en color naranja después de ser seleccionadas. En la sección “Run this command” recibirá el comando de instalación:

PyTorch

Get Started

Ecosystem

Edge

Blog

Tutorials

Docs

Resources

GitHub

Shortcuts

Prerequisites

Supported Windows Distributions

Python

Package Manager

Installation

Anaconda

pip

Verification

Building from source

Prerequisites

recommended package manager since it installs all dependencies. You can also [install previous versions of PyTorch](#). Note that LibTorch is only available for C++.

NOTE: Latest PyTorch requires Python 3.8 or later. For more details, see Python section below.

PyTorch Build	Stable (2.2.0)	Preview (Nightly)
Your OS	Linux	Mac
Package	Conda	Pip
Language	Python	C++ / Java
Compute Platform	CUDA 11.8	CUDA 12.1
		ROCm 5.7
		CPU

Run this Command:
pip3 install torch torchvision torchaudio

- Contenido de la variable a modificar: El comando de instalación debe ser asignado a la variable “command_pytorch” entre comillas, para que al ejecutar la celda se instale en base a los requisitos seleccionados:



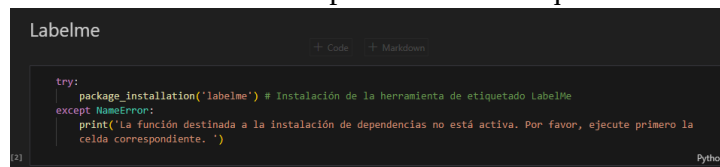
```

Dirección para instalar mediante el comando la versión de PyTorch en función de los requerimientos
computacionales: Versión PyTorch

Click here to ask Blackbox to help you code faster
command_pytorch = 'pip3 install torch torchvision torchaudio' # Comando de instalación según la version
personalizada a sus requerimientos computacionales
substrings = command_pytorch.split(" ")
try:
    subprocess.check_call(substrings)
except subprocess.CalledProcessError as e:
    print(f'Error al instalar la version de PyTorch: {e}')
  
```

5.2 Herramientas de marcado: En la sección “Instalación de herramientas de etiquetado” se encuentran los paquetes necesarios para realizar el proceso de marcado en los datos de entrenamiento y el cambio del formato para la compatibilidad con YOLO:

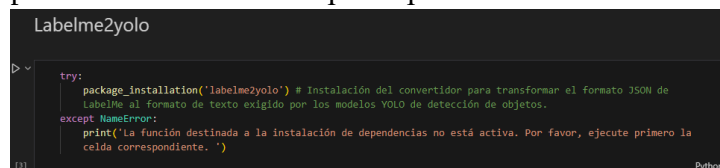
- Instalación del LabelMe para iniciar el etiquetado:



```

Labelme
try:
    package_installation('labelme') # Instalación de la herramienta de etiquetado LabelMe
except NameError:
    print('La función destinada a la instalación de dependencias no está activa. Por favor, ejecute primero la
    celda correspondiente. ')
  
```

- Instalación de LabelMe2Yolo para el cambio de formato “.json” generado por LabelMe a “.txt” aceptado por YOLO:



```

Labelme2yolo
try:
    package_installation('labelme2yolo') # Instalación del convertidor para transformar el formato JSON de
    LabelMe al formato de texto exigido por los modelos YOLO de detección de objetos.
except NameError:
    print('La función destinada a la instalación de dependencias no está activa. Por favor, ejecute primero la
    celda correspondiente. ')
  
```

6. Manejo de los datos: Para el procesamiento de las imágenes existen dos alternativas. Según la preferencia del usuario, se puede optar por separar las imágenes de manera manual o automática mediante una función.

6.1 Organizar los datos manualmente: Para separar las imágenes antes de procesarlas con LabelMe, debe utilizarse la siguiente nomenclatura:

- Datos de entrenamiento: Se requiere almacenar el 80% de las imágenes en una carpeta llamada “train”.
- Datos de validación: Es importante que el modelo valide el rendimiento con datos que no haya visto, por lo tanto, el 20% de imágenes restantes deben estar en una carpeta llamada “val”.

Advertencia: Las carpetas “train” y “val” deben ser creadas en la carpeta *main* que este destinada a guardar los datos para el uso de entrenamiento:

- La ruta de la carpeta main del modelo de marcas es: `../..../proyecto_yolo/dist/global_marks`
- Para el modelo de afectación (rayón y abolladura), la ruta es: `../..../proyecto_yolo/dist/lote2_2`



6.2 Organizar los datos automáticamente: La celda permite dividir de manera automática las imágenes, asignando un porcentaje mayor para la carpeta de “entrenamiento” y el restante para “validación”. Puede utilizarse después del etiquetado con LabelMe, queda a preferencia del usuario, si no distribuye los datos de manera manual o automática y desea pasar al etiquetado puede hacerlo, la función ofrece verificación para dividir con la misma nomenclatura los datos con los archivos JSON correspondientes:

Configuración de rutas de entrenamiento | Movimiento de datos globales etiquetados

```

# Función para separar la carpeta global de todos los datos en un 80% entrenamiento y 20%
# validación. Utilice esta función solo si no ha ordenado los datos en una carpeta para
# entrenamiento y otra para validación.

def creation_directories(route, name_folder):
    try:
        exit_route = f'{os.path.abspath(route)}\\{name_folder}'
        if not os.path.exists(f'{route}/{name_folder}'):
            os.makedirs(exit_route, exist_ok=True)
        return exit_route
    except Exception as e:
        print(f'Error in creating the folder: {e}')

def file_separation(orig_folder):
    dataset = []
    try:
        list_files = os.listdir(orig_folder)
        dataset.extend(
            i
            for i in list_files
            if i.lower().endswith(('png', '.jpg', '.jpeg', '.gif'))
        )
    except Exception as e:
        print(f'Error in file separation: {e}')
  
```

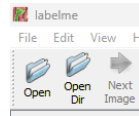
7. Etiquetado de los datos | Abrir la herramienta LabelMe: Al ejecutar la celda que permite abrir la herramienta de marcado, se mostrará en la barra de tareas el icono correspondiente. En caso contrario, se indicará el problema como un mensaje de salida:

```

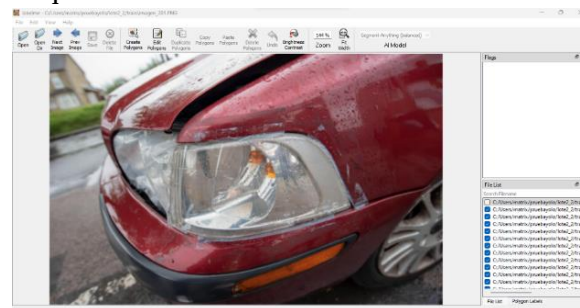
# Inicio del proceso de marcado de imágenes utilizando LabelMe
try:
    subprocess.check_call(["labelme"])
except Exception as e:
    print(f'No fue posible abrir labelme: {e}')
  
```

(4) ✓ 0.0s Python

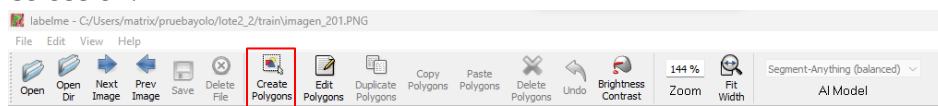
- Apertura del directorio de imágenes: En la esquina superior izquierda de la pantalla principal de LabelMe, al seleccionar el icono “Open dir” se abrirá el explorador de archivos.



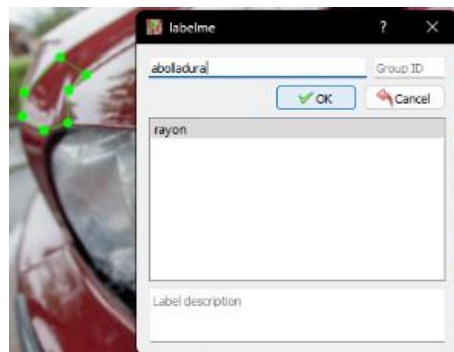
- Localización de la carpeta destinada a almacenar los datos: En la carpeta raíz del proyecto, el directorio “dist” se utiliza para almacenar datos estáticos. En él se incluyen las carpetas que contienen las imágenes a procesar (lote_2_2 y global_marks). Con el explorador de archivos debe seleccionar la ruta. De este modo, todas las imágenes contenidas en la carpeta se cargarán automáticamente en LabelMe y estarán listas para ser etiquetadas.



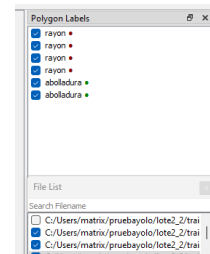
- Para etiquetar las áreas de interés en las imágenes, se utiliza la función “Create Polygons” que se encuentra ubicada en el menú superior. Con esta herramienta, es posible dibujar los contornos alrededor del área de selección.



- Una vez definida una región, aparecerá una ventana emergente donde se debe ingresar el nombre de la clase a la que pertenecerá dicha área.

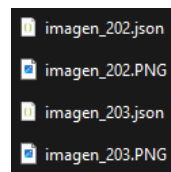


- Las clases definidas por imágenes podrán visualizarse en el panel derecho de LabelMe, justo encima de la lista de datos de la carpeta.



- Para cada imagen, es necesario repetir el proceso de clasificación de polígonos, además, se debe guardar el archivo en la opción save_as de file. Se generará automáticamente una estructura de archivos que incluirá las imágenes y el archivo JSON correspondiente, el cual contiene la información de los polígonos, así como la clase a la que pertenece.

El contenido de la carpeta debe verse de esta forma:



8. Carga de direcciones: En este paso, se utilizan rutas relativas que hacen referencia a la ubicación de un archivo en relación con la ubicación actual, en lugar de su ubicación absoluta en el sistema de archivos. Es importante destacar que al igual que el apartado “Carga de librerías”, la “Carga de rutas” es un paso esencial que debe realizarse cada vez que se inicie el proyecto. Esto asegura que todas las rutas y librerías estén actualizadas y listas para usar:

Carga rutas

```

Click here to ask Blackbox to help you code faster
current_folder= os.getcwd() # Obtener ubicación actual
main_folder= os.path.abspath(os.path.join(current_folder, "../../..")) # Carpeta main del proyecto
main_staticos= os.path.abspath(os.path.join(main_folder)+'\\dist') # Carpeta main de los archivos
estaticos
main_production= os.path.abspath(main_folder) +'\\src' # Carpeta main de los archivos listos para
producción
dir_prueba= os.path.abspath(main_folder) + "\\src\\assets" # Carpeta de imagenes prueba para el proceso
de predecir

training_structure="lote2_2" # Indica el nombre de la carpeta que tiene los archivos para el
entrenamiento (image, json)
dir_train= os.path.abspath(main_staticos) + f'\\{training_structure}'

```

[12] Python

9. Procedimiento de conversión de formato JSON a YOLO: Con el fin de garantizar la compatibilidad con los modelos YOLO para la detección de objetos o segmentación

de instancias, es necesario transformar los JSON generados en el etiquetado de clases al formato de texto (.txt):

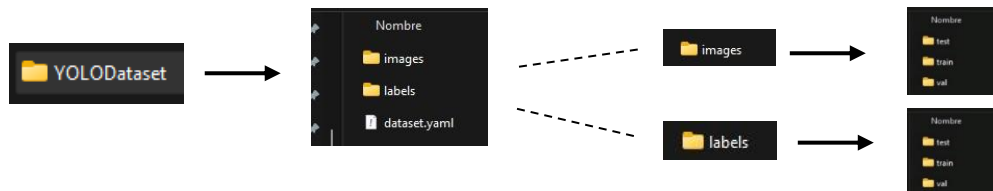
Conversión a formato YOLO

```
Click here to ask Blackbox to help you code faster
# Función para convertir la carpeta de entrenamiento y validación en formato YOLO

def convert_to_yolo_format(training_directory, validation_directory):
    try:
        subprocess.check_call(["labelme2yolo", "--json_dir", training_directory])
        print(f'Conversión completada para la carpeta {training_directory}')
        subprocess.check_call(["labelme2yolo", "--json_dir", validation_directory])
        print(f'Conversión completada para la carpeta {validation_directory}')
    except subprocess.CalledProcessError as e:
        print(f'Error al ejecutar función labelme2yolo: {e}')

convert_to_yolo_format(train_path, val_path)
```

- Como resultado de este proceso, se crearán dos carpetas con el nombre YOLODataset. Una de estas carpetas será para los datos de entrenamiento y la otra para los datos de validación.



- Dada la complejidad de la estructura de la carpeta resultante de la conversión, se ofrecen los siguientes pasos para facilitar su organización:
 - o La función inicial está diseñada para estructurar el contenido de la carpeta de conversión, extrayendo los elementos de las subcarpetas hacia las carpetas principales para su fácil acceso.

Conversión a formato YOLO | Configuración del dataset

```
# Organización de la estructura del repositorio YOLODataset (resultado de la conversión .json a formato YOLO) creado por defecto

def reorganization(path):
    try:
        new_path = os.path.abspath(os.path.join(path, "YOLODataset"))
        main_folders = os.listdir(new_path)
        for folder_instance in main_folders:
            secondary_folder_path = os.path.join(new_path, folder_instance)
            if os.path.isdir(secondary_folder_path):
                secondary_folders = os.listdir(secondary_folder_path)
                for secondary_folder in secondary_folders:
                    subfolder_secondary_path = os.path.join(secondary_folder_path, secondary_folder)
                    try:
                        if os.listdir(subfolder_secondary_path) != []:
                            for file in os.listdir(subfolder_secondary_path):
                                orig_path = os.path.join(subfolder_secondary_path, file)
                                dest_path = os.path.abspath(os.path.join(
                                    subfolder_secondary_path, "../"))
                                new_dest_path = os.path.join(dest_path, file)
                                os.rename(orig_path, new_dest_path)
                    except:
                        pass
```

- o La segunda función a ejecutar se encargará de trasladar la carpeta YOLODataset a una nueva ubicación dentro del directorio estático. Según el modelo que se utilice, la carpeta se denominará “data_affectation” para el modelo de daños o “data_marks” para el

modelo de marcas. Este procedimiento contribuye a mantener una estructura ordenada, ya que asegura que exista una carpeta dedicada exclusivamente a la carga de archivos.

```
# Función para mover las carpetas yolo

def acomodar_rutas_dataset(main_staticos, dataset_train, dataset_val):
    try:
        if not os.path.exists("data_affectation"):
            try:
                paths_segmentation= os.path.join(main_staticos, "data_affectation")
                os.makedirs(paths_segmentation, exist_ok=True)
                if os.listdir(paths_segmentation) == []:
                    shutil.move(dataset_train, os.path.join(paths_segmentation,
                                                                "YOLODataset_train"))
                    shutil.move(dataset_val, os.path.join(paths_segmentation, "YOLODataset_val"))
                    print(f'Carpetas movidas exitosamente a {paths_segmentation}')
                else:
                    print(f'Las carpetas ya se encuentran en {paths_segmentation}')
            except OSError as e:
                print(f'Error al intentar crear la carpeta en {paths_segmentation}: {e}')
            return paths_segmentation
        except TypeError as e:
            print(f'Tipo de dato incorrecto para el argumento: {e}')

segmentation_routes= acomodar_rutas_dataset(main_staticos, yolo_train_data, yolo_val_data)
```

- La última función verifica y actualiza el archivo dataset.yaml de cada carpeta YOLODataset resultante de la conversión. Durante la separación del conjunto de datos inicial en dos carpetas (train y val), puede ocurrir el caso de que algunas etiquetas se distribuyan de manera desigual, como solo etiquetas de rayón en la carpeta train o la mitad de las etiquetas de vehículo en la carpeta val. Esto resulta en la creación de dos archivos “.yaml” que contienen la configuración incompleta, por lo tanto, se busca consolidar la información en una única configuración base.

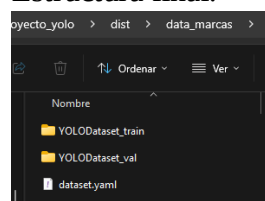
```
# Set de funciones para ordenar la estructura de los dataset individuales.

def arrange_dataset_paths(train, val):
    destination = os.path.abspath(os.path.join(train, "../.."))
    shutil.move(train, destination)
    if "dataset.yaml" in os.listdir(os.path.join(val, "../..")):
        os.remove(val)

# Función para reescribir el dataset
def rewrite_dataset(dir_train_converted, combined_list):
    with open(dir_train_converted, 'w') as new_dataset:
        new_dataset.write(
            f'train: {os.path.abspath(os.path.join(dir_train_converted, "../.."))}\n'
            f'val: {os.path.abspath(os.path.join(dir_train_converted, "../.."))}\n'
            f'\\YOLODataset_val\n'
            'test: \n'
            f'nc: {len(combined_list)}\n'
            f'names: {combined_list}\n'
        )

# Función para combinar los dos dataset generados
def total_labels(list1, list2, update_dataset_train):
    set1=set(list1)
    set2=set(list2)
    union_without_repeating = set1.union(set2)
    combined_list = list(union_without_repeating)
    rewrite_dataset(update_dataset_train, combined_list)
```

Estructura final:



9.1 Por último, se debe obtener la ruta de la estructura final que se utilizara para los procesos de entrenamiento y validación:

Ubicación de la Carpeta Preparada para el Entrenamiento

```
> # Buscar la carpeta que contiene la configuración completa para el entrenamiento
data_train= os.path.abspath(main_staticos) + "\\data_affectation\\dataset.yaml"
```

Advertencia sobre la ejecución de funciones de conversión: Es crucial ejecutar las funciones descritas solo cuando sea necesario ordenar o actualizar el conjunto de datos. Su propósito es preparar y organizar los datos para entrenamiento y validación. Si no hay cambios en los datos, basta con ejecutar la celda que permite al sistema identificar la ruta de la configuración establecida.

10. Configuración de carpetas predefinidas de YOLO: Es importante definir la ubicación de las carpetas resultantes de ejecutar cualquier tarea de entrenamiento, validación, exportación y predicción. Este enlace solo necesita establecerse una vez:



Almacena la carpeta que contiene los archivos de configuración (datasets).



Almacena los resultados de utilizar la tarea “predicción” de YOLO.



Almacena los resultados de utilizar la tarea “entrenamiento” de YOLO.

Configuración de las carpetas predeterminadas de YOLO

```
> # Modificación de las rutas de las carpetas predeterminadas de ultralytics YOLO

def set_config_ultralytics(main):
    settings.update({'datasets_dir': f'{main}\\dist', 'weights_dir': f'{main}\\src', 'runs_dir':
f'{main}\\src\\runs'})
    set_config_ultralytics(main_folder)
```

11. Selección modelo YOLOV8: Para poder realizar el primer entrenamiento se debe utilizar una versión del modelo YOLO con el conjunto de datos seleccionado por el usuario. Además, la selección de un modelo original es requerido para la detección de vehículo en la tarea de predicción.

El modelo incluye varias versiones a instalar según la preferencia del usuario.

- Identificadores para las variantes del modelo YOLOv8:

Selección versión de modelo YOLOv8

→ Tabla de variantes para la segmentación de objetos

Modelo	Nombres de archivo	Identificador
YOLOv8	yolov8n-seg.pt yolov8s-seg.pt yolov8m-seg.pt yolov8l-seg.pt yolov8x-seg.pt	n, s, m, l, x

- Ultralytics proporciona una tabla de rendimiento para el estudio de las características que amerita cada versión, proporcionadas de menor a mayor requerimiento:

- Tarea de detención:

Rendimiento

[Detección \(COCO\)](#)
[Detección \(Imágenes abiertas V7\)](#)
[Segmentación \(COCO\)](#)
[Clasificación \(ImageNet\)](#)
[Pose \(COCO\)](#)

Consulta los Documentos de Detección para ver ejemplos de uso con estos modelos entrenados en COCO, que incluyen 80 clases preentrenadas.

Modelo	tamaño (pixels)	mAP _{val} 50-95	Velocidad CPU ONNX (ms)	Velocidad A100 TensorRT (ms)	parámetros (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

- Tarea de segmentación:

Rendimiento

[Detección \(COCO\)](#)
[Detección \(Imágenes abiertas V7\)](#)
[Segmentación \(COCO\)](#)
[Clasificación \(ImageNet\)](#)
[Pose \(COCO\)](#)

Consulta Segmentation Docs para ver ejemplos de uso con estos modelos entrenados en COCO, que incluyen 80 clases preentrenadas.

Modelo	tamaño (pixels)	mAP _{box} 50-95	mAP _{mask} 50-95	Velocidad CPU ONNX (ms)	Velocidad A100 TensorRT (ms)	parámetros (M)	FLOPs (B)
YOLOv8n-seg	640	36.7	30.5	96.1	1.21	3.4	12.6
YOLOv8s-seg	640	44.6	36.8	155.7	1.47	11.8	42.6
YOLOv8m-seg	640	49.9	40.8	317.0	2.18	27.3	110.2
YOLOv8l-seg	640	52.3	42.6	572.4	2.79	46.0	220.5
YOLOv8x-seg	640	53.4	43.4	712.1	4.02	71.8	344.1

- Librería que se encargará de la descarga del modelo en el equipo:

✕
 Instalación del paquete para la gestión de descarga de los modelos

```

▶ package_installation('gdown') # Librería que permitirá descargar las versiones de YOLOv8
[ ]

```

11.1 Modelo original: Al ejecutar la celda, aparecerá un cuadro de diálogo en el que se debe especificar el identificador correspondiente a la versión que se desea instalar o utilizar.

Modelo original

```
def get_model_version():
    available_versions= ['n', 's', 'm', 'l', 'x'] + yolo@n.pt , yolo@s.pt , yolo@m.pt , yolo@l.pt , yolo@x.pt
    control= True
    while control:
        version_selection= input("Ingrese la versión del modelo que desea utilizar, entre las disponibles están: n, s, m, l, x. Advertencia: Si no desea
        instalar ninguna ingrese la palabra 'salir' ").lower()
        if version_selection == "salir":
            id_version=None
            control=False
        elif version_selection in available_versions:
            id_version= f'yolo@{version_selection}-seg.pt'
        else:
            print('Version de modelo inexistente, Ingrese nuevamente! ')
        return id_version

def model_installation(url, version_name):
    path_save = os.path.abspath(main_folder) + "\\dist\\model_version\\" + version_name
    if os.path.exists(path_save):
        print(f'Se está utilizando la versión "{version_name}" del modelo. ')
    else:
        try:
            # ... (código de instalación) ...
```

11.2 Modelo pre-entrenado: Para utilizar el modelo pre-entrenado, es necesario haber completado el modo de entrenamiento con un modelo original. En caso de que existan experimentos la función seleccionará automáticamente el último entrenamiento disponible. De lo contrario, aparecerá un mensaje de advertencia indicando que no existen experimentos.

- Para hacer seguimiento de los modelos pre-entrenados la ruta que almacena los experimentos es:
“.../.../proyecto_yolo/src/<srcv_version>/<tipo_modelo>/<carpeta_experimentos>”.

Modelo pre-entrenado

```
# Función para obtener el último entrenamiento

def get_last_training(dir_src, det_type):
    try:
        yolo_model_folders= os.listdir(dir_src)
        for folder_name in yolo_model_folders:
            if folder_name == det_type:
                base_path = os.path.abspath(os.path.join(dir_src, folder_name))
                files= os.listdir(base_path)
                last_experiment = sorted(files)[-1]
                model= os.path.join(base_path, last_experiment, "weights", "best.pt")
                print(f'Los pesos del experimento que se van a utilizar son {model}')
                return model
            else:
                print(f'No existe ningún experimento de entrenamiento en la carpeta {dir_src}')
                return None
    except FileNotFoundError as e:
        print(f'Error: {e}')

# Llamado para obtener el último entrenamiento del modo segmentación
try:
```

Advertencia: La tarea de predicción requiere la utilización de los dos tipos de modelo (original y pre-entrenado), por lo tanto, es recomendable ejecutar las celdas que obtienen dichos modelos cada que se inicie el script.

A partir de aquí se puede utilizar las tareas de YOLOv8 mencionadas (ENTRENAMIENTO, EXPORTACIÓN, VALIDACIÓN Y PREDICCIÓN).

Para llevar a cabo las tareas de YOLO, se sigue una secuencia de pasos. En primer lugar, se configuran los hiperparámetros, seguido a esto, se carga un modelo YOLO y se inicia la tarea utilizando la instancia del modelo. Con el fin de mantener una estructura en el código y evitar redundancias, se ha creado una clase que contiene los pasos clave para la ejecución de cada tarea:

- Se identificaron los hiperparámetros comunes entre las configuraciones de ejecución para cualquier tarea de YOLO.
- Cada tarea tiene un diccionario con configuración privada y otro para la modificación personalizada por el usuario de dichos valores preestablecidos.
- Se incluyen funciones de control y seguridad para evitar situaciones en donde se configure un parámetro que no pertenece a dicha tarea y no permita su ejecución exitosa.
- El diccionario privado es el objeto modificable en cada llamada a la clase, es decir, incluye los valores de los hiperparámetros que el usuario desea modificar, así como los argumentos comunes con valores preestablecidos por tarea.

Configuración de hiperparámetros

```
class Custom_settings:
    def __init__(self, name, base_setup, kwargs, project='', imgsz=640, device='cpu', save=True):
        project: str; name: str; device: str
        imgsz: int # También puede ser representado como tupla -> (ancho, alto), ejemplo: imgsz=(640, 480).
        save: bool

        self.project = project # Permite englobar los experimentos en una carpeta base.
        self.name = name # Define el nombre de los experimentos.
        self.imgsz = imgsz # Indica la dimensión en píxeles de las imágenes de entrada.
        self.device = device # Establece el dispositivo de ejecución (CPU o GPU).
        self.save = save # Controla la conservación de los resultados en el disco duro.
        self.base_setup = base_setup # Diccionario base de configuración por tarea.
        self.changes = kwargs # Diccionario de modificación de parámetros definidos por el usuario.

    def _remove_parameters(self):
        try:
            keys_to_remove = [param for param in self.changes.keys() if param not in self.base_setup] # Obtiene los parámetros que no pertenecen a la configuración base de tarea
            print(f'Parameters not available for the task: {keys_to_remove}')

            for invalid_parameter in keys_to_remove:
                del self.changes[invalid_parameter] # Elimina del diccionario de modificación los parámetros no válidos
        except Exception as e:
            print(e)
```

A excepción de la tarea predicción, se abrirá una ventana emergente donde se consultará sobre el tipo de modelo que desea emplear para realizar la tarea:

```
def model_menu():
    models_types = ('1':'original', '2':'entrenado') # Almacena los tipos de modelo disponibles para ejecutar las tareas de YOLO
    while True:
        try:
            option = input(f'Ingrese el identificador del modelo que desea utilizar para la ejecución de la tarea: {", ".join([f"{id_model}: {value}" for id_model, value in models_types.items()])}. Una vez desea cancelar el proceso ingrese la palabra "salir".').lower()
            if option == "salir":
                break
            if option in models_types:
                if models_types[option] == "original": # Realiza la consulta mediante el identificador del modelo
                    return model_orig # Retornará la versión del modelo original elegida con anterioridad
                else:
                    return model_affectation
            else:
                print('Character not valid, try again. ')
        except Exception as e:
            print(f'The model is not active, go to the original model or trained model section and run the cell: {e}')
    return None
```

Tarea de entrenamiento:

```
Entrenamiento

class Train:
    def __init__(self, instance_model, data, **kwargs):
        data : str; instance_model : str

        self.data = data # Incluye el archivo de configuración (dataset.yaml) para el entrenamiento de los datos.
        self.custom_settings = kwargs # Hiperparámetros de configuración adicionales.
        self.instance_model = instance_model # Instancia del modelo YOLO seleccionado.
        self.training_settings = { # Configuración de atributos permitidos para la tarea de entrenamiento.
            'task': 'segment', # Establece el tipo de salida de la tarea; detect (detección de objetos) y segment (segmentación de instancias).
            'epochs': 30, # Número de iteraciones(pasos) al conjunto de entrenamiento.
            'patience': 30, # Número de paciencia(epocas de espera) cuando no se observe mejora de aprendizaje.
            'optimizer': 'auto', # Define el algoritmo de optimización para el modelo de segmentación YOLO. Su elección ajusta los pesos del modelo durante
            el entrenamiento y es crucial para la velocidad y rendimiento. Puede tomar valores como 'SGD', 'Adam', 'Adamax', 'AdamW', 'Nadam', 'Radam',
            'RMSprop' y 'auto', este último selecciona automáticamente el optimizador más adecuado a la tarea segmentación de objetos.
            'workers': 1,
            'plots': True,
            'verbose': False,
            'rect': False,
            'cos_lr': False,
            'fraction': 1.0,
            'exist_ok': False,
            'overlap_mask': True,
            'exist_ratio': 1
        }
```

Tarea de exportación: En la sección “Configuración para la exportación” se proporciona información de consulta sobre las características que utiliza el formato a exportar:

Configuración para la exportación

A continuación se muestra una tabla de referencia para exportar un modelo YOLOv6 entrenado en la tarea segmentación de instancias. Tenga en cuenta que configurar el parámetro de `format` es fundamental para el proceso de exportación. Antes de continuar, asegúrese de verificar y ajustar estos valores a sus requisitos específicos:

Formatos	Asignación	Extensión	Hiperparámetros	Descripción
PyTorch	-	yolov6n.pt	-	Modelo en formato PyTorch
TorchScript	"torchscript"	yolov6n.torchscript	imgsz, optimize	Simplifica la implementación de modelos PyTorch en entornos de producción y aplicaciones eficientes, mejorando la portabilidad y el rendimiento al

```
Exportación

# Opción valor por defecto flexible
class Settings_export(Custom_settings):
    def __init__(self, id_format, imgsz, base_setup, kwargs):
        self.id_format = id_format
        self.changes = kwargs
        self.base_setup = base_setup
        super().__init__(imgsz, base_setup, kwargs)

    def _remove_parameters(self):
        try:
            keys_to_remove = [param for param in self.changes.keys() if param not in self.base_setup[self.id_format]]
            print(f'Parameters not available for the task: {keys_to_remove}')

            for invalid_parameter in keys_to_remove:
                del self.changes[invalid_parameter] # Remueve los parametros no validos
        except Exception as e:
            print(f'error(e)')

    def set_apply_defaults(self):
        try:
            self._remove_parameters()
            parameters_base = (**self.changes, 'imgsz': self.imgsz)
            self.changes.update(parameters_base)
            print(f'Configuration included: "{", ".join([f"({k})={v}" for k, v in self.changes.items()])"')
        except Exception as e:
            print(f'error(e)')
```

El diccionario de configuración predeterminado en la clase de exportación contiene la información proporcionada en la tabla. Es crucial mantener la estructura para evitar problemas de argumentos no válidos para el formato.

```
class Exportacion():
    def __init__(self, model, format, **kwargs):
        self.model = model # dirección del modelo a exportar
        self.format = format # nombre del formato de exportación seleccionado
        self.custom_settings = kwargs # diccionario con el valor de hiperparámetros a modificar
        self.formats_config = { # configuración establecida por defecto, en la forma -> formato : (hiperparámetro = valor)
            "torchscript": ('optimize': False),
            "onnx": ('half': False, 'dynamic': False, 'simplify': False, 'opset': None),
            "openvino": ('half': False, 'int8': False), # requirements: Ultralytics requirement ['openvino>2024.0.0']
            "engine": ('half': False, 'dynamic': False, 'simplify': False, 'workspace': 4.0), # Invalid CUDA 'device=0' requested. Use 'device=cpu' or pass
            valid CUDA device(s) if available, i.e. 'device=0' or 'device=0,1,2,3' for Multi-GPU.
            "coreml": ('half': False, 'int8': False, 'nms': False), # requirements: Ultralytics requirement ['coremltools>7.0']
            "saved_model": ('keras': False, 'int8': False), # requirements: Ultralytics requirements ['tf_keras', 'onnx2tf>1.17.5,<=1.22.3', 'sng4onnx>1.0.1',
            'onnxslim>0.1.28', 'onnx_graphsurgeon>0.3.26', 'tflipe_support']
            "tflipe": ('half': False, 'int8': False), # requirements: Ultralytics requirement: ['tf_keras', 'onnx2tf>1.17.5,<=1.22.3', 'sng4onnx>1.0.1',
            'onnxslim>0.1.28', 'onnx_graphsurgeon>0.3.26', 'tflipe_support']
            "edgetpu": (), # Edge TPU export only supported on Linux. See https://coral.ai/docs/edgetpu/compiler
            "tfjs": (), # requirements: Ultralytics requirements ['tf_keras', 'onnx2tf>1.17.5,<=1.22.3', 'sng4onnx>1.0.1', 'onnxslim>0.1.28',
            'onnx_graphsurgeon>0.3.26', 'tflipe_support']
            "paddle": (), # requirements: Ultralytics requirements ['paddlepaddle', 'x2paddle']
            "ncnn": ('half': False) # requirements: Ultralytics requirement ['ncnn']
        }

        self.config = Settings_export(id=format=self.format, base_setup=self.formats_config, kwargs=self.custom_settings, imgs=640)
        self.config.set_apply_defaults()

    def export_model(self):
```

Tarea de validación:

```
Validación

class Val:
    def __init__(self, instance_model, data, **kwargs):
        self.instance_model = instance_model
        self.data = data
        self.custom_settings = kwargs
        self.validation_settings = {
            'save_json': True,
            'save_hybrid': True,
            'conf': 0.5,
            'max_det': 10,
            'plots': True,
            'rect': False,
            'iou': 0.6
        }

        self.config = Custom_settings(name="validation_affectation_srcv5", base_setup=self.validation_settings, kwargs= self.custom_settings)
        self.config.set_apply_defaults()

    def validation(self): # Validación
        try:
            self.config.loading_model(self.instance_model).val(data= self.data, **self.validation_settings)
        except Exception as e:
            print(f'Error in the validation process: {e}')

        try:
```

Tarea de Predicción:

Es posible emplear diversas fuentes para el proceso de predicción, información proporcionada en la sección “Configuración de fuentes”. Si desea obtener mayor referencia sobre cómo implementarlas, visite el sitio oficial de Ultralytics:

[Predict - Ultralytics YOLO Docs](#)

Configuración de fuentes

Para utilizar múltiples fuentes de datos al realizar predicciones con el modelo, se requiere que se ajuste el parámetro 'source' a sus necesidades, tal como se indica en la siguiente tabla:

Fuentes	Asignación	Tipo	Notas
image	'image.jpg'	str or Path	Archivo que contiene una única imagen.
URL	'https://ultralytics.com/images/bus.jpg'	str	Dirección que especifica la ubicación de una imagen en la web.
screenshot	'screen'	str	El sistema captura la imagen actualmente visible en la pantalla y la utiliza como entrada para el modelo.
PIL	Image.open('im.jpg')	PIL.Image	Utilizado para cargar imágenes en formato HWC (altura, ancho, canales) con canales RGB (rojo, verde y azul) mediante la biblioteca Python Imaging Library (PIL).
OpenCV	cv2.imread('im.jpg')	np.ndarray	Permite la lectura de una imagen desde un archivo en formato HWC con canales BGR (azul, verde, rojo) utilizando la biblioteca OpenCV, almacenando la imagen como un array de NumPy.

```
Predicción

class Predict:
    def __init__(self, name, isinstance_model, data, classes, save_value, **kwargs):
        self.name = name
        self.isinstance_model = isinstance_model
        self.data = data
        self.classes = classes
        self.save_value = save_value
        self.custom_settings = kwargs
        self.prediction_settings = {
            'conf': 0.4,
            'max_det': 10,
            'save_txt': False,
            'visualize': False,
            'line_width': None,
            'iou': 0.6,
            'vid_stride': False,
            'stream_buffer': False,
            'save_frames': False,
            'augment': False, # augment=True -> Error when executing the prediction task: 'NoneType' object is not subscriptable
            'save_crop': False,
            'show': False,
            'save_conf': True,
            'agnostic_nms': True,
            'retina_masks': False
        }
    }
```

Función Calculo del Área: Para poder utilizar las funciones de área es necesario haber ejecutado la tarea de predicción debido a que se necesitan los resultados de las máscaras segmentadas y las cajas de detección.

```
Función de área para mascarar segmentadas
Dataframe con las áreas totales

def calculate_area(vertices):
    total_mask_area = 0 # Se inicializa una variable acumuladora para almacenar la suma total del área de cada máscara por clase en una imagen.
    for point in vertices: # Recorre los puntos de contorno que se encuentran separados por medio de una lista para cada región segmentada.
        x_coordinate = point[0, 0] # Asigna todas las filas correspondientes a la primera columna como coordenada X del formato de masks.xy que entrega por medio de arreglos los vértices (x, y) en píxel.
        y_coordinate = point[0, 1] # Asigna todas las filas correspondientes a la segunda columna como coordenada Y.
        mask_area = np.round(0.5 * abs(np.dot(x_coordinate, np.roll(y_coordinate, 1)) - np.dot(y_coordinate, np.roll(x_coordinate, 1)))) # Almacena el resultado del área por máscara, basándose en la fórmula Shoelace o lazada de Gauss. Utiliza la suma y la resta de los productos cruzados de las coordenadas de los vértices del polígono; el término np.roll(y, 1) y np.roll(x, 1) crean versiones desplazadas circularmente, lo cual es esencial para que np.dot() calcule el producto punto de los vértices del polígono. La diferencia entre los productos cruzados se multiplicará por 1/2 tomando el valor absoluto.
        Individual_mask_area.append(mask_area) # Añade a la lista el resultado del área.
    return 0 # Cuando no exista resultados de máscara se retorna como valor 0, que indica no haber área.
return total_mask_area
```

```
Dataframe con las áreas individuales

def calculate_area(vertices):
    Individual_mask_area = [] # Se inicializa una lista vacía para guardar el área individual de cada objeto segmentado.
    if vertices: # Verifica si existe resultados de máscara.xy que proporciona los datos en formato píxel.
        for point in vertices: # Recorre los puntos de contorno que se encuentran separados por medio de una lista para cada región segmentada.
            x_coordinate = point[0, 0] # Asigna todas las filas correspondientes a la primera columna como coordenada X del formato de masks.xy que entrega por medio de arreglos los vértices (x, y) en píxel.
            y_coordinate = point[0, 1] # Asigna todas las filas correspondientes a la segunda columna como coordenada Y.
            mask_area = np.round(0.5 * abs(np.dot(x_coordinate, np.roll(y_coordinate, 1)) - np.dot(y_coordinate, np.roll(x_coordinate, 1)))) # Almacena el resultado del área por máscara, basándose en la fórmula Shoelace o lazada de Gauss. Utiliza la suma y la resta de los productos cruzados de las coordenadas de los vértices del polígono; el término np.roll(y, 1) y np.roll(x, 1) crean versiones desplazadas circularmente, lo cual es esencial para que np.dot() calcule el producto punto de los vértices del polígono. La diferencia entre los productos cruzados se multiplicará por 1/2 tomando el valor absoluto.
            Individual_mask_area.append(mask_area) # Añade a la lista el resultado del área.
    else:
        return 0 # Cuando no exista resultados de máscara se retorna como valor 0, que indica no haber área.
return Individual_mask_area
```